

Τεχνικές Βελτιστοποίησης Project 2025 - 2026

Ρούσος Σταμάτης

26 Ιανουαρίου 2026

1 Εισαγωγή

Στο πρόβλημα αυτό, θέλουμε να προσεγγίσουμε τη συνάρτηση

$$y = f(u_1, u_2) = \sin(u_1 + u_2)\sin(u_2^2)$$

$$u_1 \in [-1, 2],$$

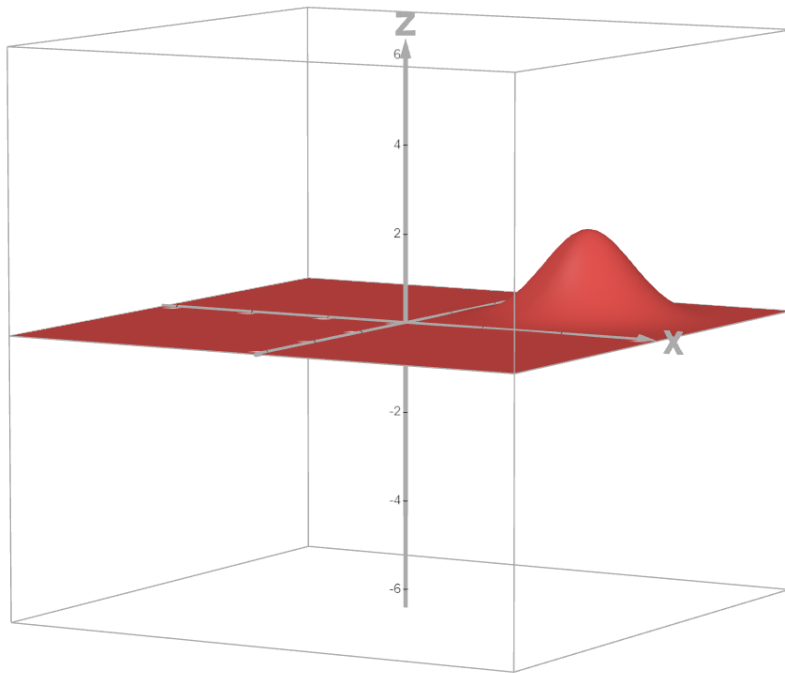
$$u_2 \in [-2, 1]$$

χρησιμοποιώντας το πολύ 15 γκαουσιανές συναρτήσεις ($K \leq 15$) της μορφής:

$$G_i(u_1, u_2) = e^{-\left(\frac{(u_1 - c_{1,i})^2}{2\sigma_{1,i}^2} + \frac{(u_2 - c_{2,i})^2}{2\sigma_{2,i}^2}\right)}$$

όπου $c_{1,i}$, $c_{2,i}$ τα κέντρα της i Γκαουσιανής ως προς τις μεταβλητές u_1 , u_2 , και $\sigma_{1,i}$, $\sigma_{2,i}$ οι διασπορές τους αντιστοίχα.

Μπορούμε να φανταστούμε τη Γκαουσιανή συνάρτηση ως μια κάμψα στο τρισδιάστατο επίπεδο, με κέντρο το σημείο $(c_{1,i}, c_{2,i})$ όπου βρίσκεται η κορυφή της κάμψας, και οι διασπορές δείχνουν πόσο απλώνεται στο επίπεδο καθώς απομακρυνόμαστε από το κέντρο (πόσο αποτομα πεφτει).



Θελουμε λοιπον να προσεγγισουμε τη συναρτηση f χρησιμοποιωντας ενα γραμμικο συνδυασμο τετοιων Γκαουσιανων συναρτησεων, δηλαδη:

$$\hat{y} = \sum_{i=0}^K w_i G_i(u_1, u_2) \approx y$$

οπου ως G_0 οριζω $G_0 = 1$, οποτε:

$$\hat{y} = w_0 + \sum_{i=1}^K w_i G_i(u_1, u_2) \approx y$$

Το w_0 ειναι ουσιαστικα το bias το οποιο επιτρεπει στη συναρτηση να δινει μια τιμη ακομα και οταν ολες οι Γκαουσιανες ειναι ανενεργες. Στην εργασια αυτη, θα λυσουμε το παραπανω προβλημα φτιαχνοντας ενα **γενετικο αλγοριθμο (GA)**.

2 Γενετικοί Αλγόριθμοι

Οι Γενετικοι Αλγοριθμοι κατα βαση λειτουργουν ως εξης:

1. Initial Population

2. Fitness Function
3. Selection (Select parents)
4. Crossover + mutation (Create kids)
5. Repeat Steps 3-4 until new population is created
6. Replace old population with new
7. Repeat Steps 2-6 until stop criterion

Λίγο πιο αναλυτικά:

1. Ξεκινάμε από έναν τυχαίο αρχικό πληθυσμό (population). Ο πληθυσμός αποτελείται από άτομα τα οποία λέγονται χρωμοσώματα (chromosomes). Ένα χρωμοσώμα, αποτελείται από διάφορα γονίδια (genes) καθένα από τα οποία είναι μια από τις παραμέτρους που ψαχνούμε. Κάθε χρωμοσώμα του πληθυσμού, είναι ένας διαφορετικός συνδυασμός από genes. Μπορούμε να φανταστούμε κάθε άτομο ως σημεία στο χώρο αναζήτησής μας.
2. Ο αλγόριθμος χρησιμοποιεί μια συνάρτηση ικανότητας (Fitness Function) με την οποία υπολογίζει την ικανότητα κάθε ατόμου. Αναλογα το πρόβλημα, όσο πιο μεγάλη/μικρή είναι η τιμή της συνάρτησης ικανότητας (fitness value) ενός ατόμου, τόσο πιο ικανό είναι.
3. Ως πρώτο βήμα, υπολογίζει τις fitness values όλων των ατόμων του αρχικού πληθυσμού.
4. Στη συνέχεια, επιλέγει με διάφορες μεθόδους ορισμένα άτομα από τον πληθυσμό ως γονείς (συνήθως 2 για κάθε παιδί).
5. Χρησιμοποιεί αυτούς τους γονείς για να δημιουργήσει παιδιά, τα οποία αναλογα με τη μέθοδο, θα είναι παραλλαγές των γονέων. Επαναλαμβάνει τα βήματα 3-4 και δημιουργεί παιδιά,

μεχρι ο νεος πληθυσμος να εχει τοσα ατομα οσα με πριν. Αντικαθιστα τον παλιο πληθυσμο με τον νεο που εφτιαξε.

6. Τελος, επαναλαμβανει τα βηματα 2-6 μεχρι να ικανοποιηθει το κριτηριο τερματισμου που θεσαμε στην αρχη (π.χ αριθμος γενιων (generations)).

3 Γενετικός Αλγόριθμος στο πρόβλημα μας

3.1 Παράμετροι

Οι παραμετροι του προβληματος μας ειναι:

$$[c_{11}, c_{12}, \dots, c_{1k} | c_{21}, c_{22}, \dots, c_{2k} | \sigma_{11}, \sigma_{12}, \dots, \sigma_{1k} | \sigma_{21}, \sigma_{22}, \dots, \sigma_{2k}] \quad (1)$$

οπου:

c_{1k} : Το κεντρο c_1 της k-οστης Γκαουσιανης

c_{2k} : Το κεντρο c_2 της k-οστης Γκαουσιανης

σ_{1k} : Η διασπορα σ_1 της k-οστης Γκαουσιανης

σ_{2k} : Η διασπορα σ_2 της k-οστης Γκαουσιανης

Ψαχνουμε λοιπον 4K παραμετρους, οπου K το πληθος των Γκαουσιανων που θα χρησιμοποιησουμε, προκειμενου να προσεγγισουμε τη συναρτηση που θελουμε. Η (1) δειχνει τη μορφη των χρωμοσωματων μας. Καθενα απο τα $c_{11}, c_{12}, \dots$ ειναι ενα gene. Επομενως τα ατομα του πληθυσμου μας θα ειναι διαφορετικοι συνδυασμοι αυτων των genes-παραμετρων. Στην υλοποιηση μου, εθεσα το K ισο με το μεγαιστο επιτρεπτο οριο Γκαουσιανων, δηλαδη 15, και για να πετυχω την ζητουμενη χαμηλη πολυπλοκοτητα, προσθεσα ποινη στη fitness value για τις ενεργες Gaussian, οπως θα δουμε παρακατω. Συνεπως, απο δω και μπρος καθε χρωμοσωμα θα αποτελεται απο $4 * 15 = 60$ genes - παραμετρους.

3.2 Αρχικός Πληθυσμός

Ο αρχικος πληθυσμος δημιουργηθηκε χρησιμοποιωντας την εντολη rand που παραγει αριθμους στο (0,1) και ακολουθουν την ομοιομορφη κατανομη, με τετοιο τροπο ομως που οι παραμετροι να βρισκονται εντος οριων. Ως ορια, για τα κεντρα c_1, c_2 εθεσα τα ιδια

με των u_1, u_2 και για τις διασπορες το (0.15,1.5) για να μην είναι ουτε πολυ μικρες ουτε πολυ μεγαλες.

3.3 Συνάρτηση Ικανότητας (Fitness Function)

Ως συναρτηση ικανοτητας, ορισα την:

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda N_{active}$$

οπου:

- θ : Το διανυσμα των παραμετρων-genes, δηλαδη ενα ατομο-χρωμοσωμα. Στο εξης θα το λεω θ .
- $\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \text{MSE}$ το μεσο τετραγωνικο σφαλμα, δηλαδη ο μεσος ορος των τετραγωνων των σφαλματων για καθε δειγμα i , οπου σφαλμα, το ποσο απεχει η τιμη της πραγματικης συναρτησης απο την προσεγγιστικη για το δειγμα i : $e_i = y_i - \hat{y}_i$.
- λN_{active} η ποινή που προσθετω στη fitness function για τις ενεργες Γκαουσιανες. Η ποινή είναι αναλογη της σταθερας λ την οποια ορισα εγω ιση με $\lambda = 0.0003$ και του πληθους των ενεργων Γκαουσιανων N_{active} . Τις ενεργες Γκαουσιανες τις ορισα με ενα σχετικο τροπο, να είναι αυτες που τα βαρη τους w_k ικανοποιουν: $|w_k| > \rho * \max(|w|)$, δηλαδη να ξεπερνουν κατα ρ φορες, το μεγαιστο απολυτο βαρος, οπου $0 < \rho \ll 1$. Π.χ αν $w = [-3, -1.3, 0.7, 2.5]$ και $\rho = 0.05$ τοτε ενεργα είναι αυτα που ικανοποιουν: $|w| > 0.05 * 3 = 0.15$ δηλαδη ολα. Στον αλγοριθμο μου, εθεσα $\rho = 0.05$. Επομενως, ανενεργες Γκαουσιανες θα είναι αυτες που εχουν μικρο σχετικο βαρος (ως προς το μεγαιστο κατα απολυτη τιμη), διοτι τοτε επηρεαζουν ελαχιστα-ασημαντα την εξοδο.

Η ποινή που εβαλα, κανει την fitness function πιο μεγαλη οσο περισσοτερες ενεργες Γκαουσιανες χρησιμοποιει ο αλγοριθμος. Ετσι,

ωθει τον GA να επιλεξει θ που οδηγει σε λιγοτερες ενεργες Γκαουσιανες, για να πετυχει μικροτερο fitness value. Για παραδειγμα, αν δυο διαφορετικα χρωμοσωματα (θ) δινουν κοντινα MSE, τοτε αν το ενα χρησιμοποιει πολυ λιγοτερες Γκαουσιανες, θα επιλεξει αυτο, γιατι θα δινει μικροτερο fitness value. Ετσι πετυχα την χαμηλη πολυπλοκοτητα.

3.4 Επιλογή

Η επιλογη εγινε με τη μεθοδο tournament selection, με tournament size = 3. Στη τεχνικη αυτη διαλεγονται τυχαια 3 ατομα, συγκρινονται τα fitness values τους, και επιλεγεται το ατομο με την μικροτερη. Ετσι, για την επιλογη δυο γονεων, καλειται 2 φορες η συναρτηση selection tournament.

3.5 Crossover

Στη συνεχεια, η δημιουργια παιδιων εγινε με τη μεθοδο crossover blx, με πιθανοτητα 0.9, + mutation. Σε περιπτωση που δεν γινει crossover, ορισα το παιδι να ειναι ιδιο με τον πρωτο γονεα του. Σε περιπτωση που γινει crossover, γινεται με βαση τη σχεση:

$$\text{child} = [\dots, \min(p_1(i), p_2(i)) - ad(i) + r(1 + 2a)d(i), \dots]$$

οπου:

- i το i -οστο gene,
- p_1, p_2 τα χρωμοσωματα των γονιων
- $r \sim U(0, 1)$,
- $d(i) = \max(p_1(i), p_2(i)) - \min(p_1(i), p_2(i))$

Ετσι, $\text{child}(i) \sim U(\min(i) - ad(i), \max(i) + ad(i))$.

3.6 Mutation

Το mutation, εγινε με μια πιθανοτητα 0.07. Με αυτη την πιθανοτητα, καθε gene μεταλλασσεται προσθετοντας Gaussian θορυβο.

Συγκεκριμένα:

$$\theta(i)' = \theta(i) + r * step = \theta(i) + \varepsilon$$

οπου:

- $r \sim N(0, 1)$
- $step = \rho * range(\theta(i))$, με ρ μια σταθερα κλιμακωσης και $range(\theta(i))$ να ειναι το ευρος των τιμων που μπορει να παρει το gene $\theta(i)$, π.χ $range(c_1) = 2 - (-1) = 3$.
- $\varepsilon \sim N(0, step^2)$

Ο λογος που ορισα το step να ειναι αναλογο του ευρους των τιμων της καθε παραμετρου, ειναι γιατι ενα βημα μεταλλαξης π.χ 0.5 ειναι διαφορετικο για τις παραμετρους c και διαφορετικο για τη διασπορα σ . Αν τελος δεν γινει μεταλλαξη του gene, παραμενει οπως ηταν.

Τα βηματα 3.4 - 3.6 επαναλαμβανονται μεχρι να δημιουργηθουν ολα τα παιδια. Αυτα περνανε στο νεο πληθυσμο, μαζι με τους 3 καλυτερους (επιζωντες) του προηγουμενου. Αυτη η τεχνικη (keep few survivors) ονομαζεται elitism και εξασφαλιζει οτι δεν χανονται οι καλυτερες λυσεις. Σημειωνω οτι μετα το crossover και τη μεταλλαξη, καλεσα τη συναρτηση clamp bounds, για να επαναφερει ολα τα genes των παιδιων μεσα στα επιτρεπτα ορια.

3.7 Κριτήριο Τερματισμού

Το κριτηριο τερματισμου του αλγοριθμου που ορισα ειναι οι 800 γενιες.

3.8 Ανακεφαλαίωση GA λογικής

Ο γενετικος αλγοριθμος μας λοιπον, προσπαθει να ελαχιστοποιησει την fitness function μεσω αναζητησης στο χωρο των παραμετρων. Ο πληθυσμος ειναι υποψηφιες λυσεις και λειτουργει ως σημεια αναζητησης. Σε καθε γενια, αναπαραγουμε νεο πληθυσμο με τετοιο τροπο που να επιβιωνουν οι καλυτεροι και να παραγονται συνεχως καλυτερα ατομα - καλυτερες λυσεις. Ετσι, η αναζητηση γινεται ολο και καλυτερη, ο αλγοριθμος βελτιωνεται σε καθε γενια και τελικα συγκλινει στην βελτιστη λυση. Προσοχη! Λογω της

τυχαioτητας που υπαρχει στον αλγοριθμο (στην επιλογη γονιων, στη δημιουργια παιδιων κ.λπ) ειναι πιθανο καποιες φορες οι επομενες γενιες να παραξουν πληθυσμο χειροτερο του προηγουμενου. Ωστοσο εμπειρικά εχουμε συμπερανει πως ο αλγοριθμος τελικα θα συγκλινει σε καλες λυσεις.

3.9 Δημιουργία Δεδομένων

Οσον αφορα τη δημιουργια δεδομενων, δημιουργησα τυχαia δεδομενα χρησιμοποιωντας τη συναρτηση rand, εντος των οριων. Συγκεκριμενα, δημιουργησα 600 training data points για εκπαideυση, 200 validation data points, και 200 test points. Το validation set, το εφτιαξα για να αξιολογω το μοντελο μου οσο το εκανα tuning, δηλαδη οσο πειραζα τις υπερπαραμετρους (population size, tournament size, training data, generations, mutation rate/scale κλπ) για να βρω καλυτερες λυσεις. Το test set πρεπει να μεινει αθικτο μεχρι να βρω το καλυτερο μοντελο μου, διοτι αν εκανα tuning με βαση το test set, στην ουσια προσπαθω να κανω το μοντελο μου αριστο στα συγκεκριμενα δεδομενα, οποτε μπορει να γινει καλο σε αυτα, αλλα να μη γενικευει καλα σε διαφορετικα, αγνωστα δεδομενα. Χανεται δηλαδη η εννοια της αξιολογησης του μοντελου σε αγνωστα δεδομενα.

3.10 Υπολογισμός βαρών w_i

Τελος, να αναφερω οτι τα βαρη w_i τα υπολογιζω στο solve w LS. Αρχικα, τα θεωρουσα κι αυτα ως παραμετρους, οποτε ειχα $5K=5*15=75$ παραμετρους και ο αλγοριθμος ηταν πιο αργος και εδινε χειροτερα αποτελεσματα. Οποτε, δοκιμασα να μη τα θεωρησω ως παραμετρο, αλλα να τα υπολογιζω ως εξης:

- Για καθε θ , εχω:

$$\hat{y} = \hat{f}(u_1, u_2) = w_0 + \sum_{k=1}^K w_k G_k(u_1, u_2)$$

Για καθε δειγμα i:

$$\hat{y}_i = \begin{bmatrix} 1 & G_1(u_i) & \cdots & G_K(u_i) \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_K \end{bmatrix}$$

Αρα:

$$\hat{\mathbf{y}} = \underbrace{\begin{bmatrix} 1 & G_{1,1} & \cdots & G_{1,K} \\ 1 & G_{2,1} & \cdots & G_{2,K} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & G_{N,1} & \cdots & G_{N,K} \end{bmatrix}}_{\Phi \in \mathbb{R}^{N \times (K+1)}} \underbrace{\begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_K \end{bmatrix}}_{\mathbf{w} \in \mathbb{R}^{(K+1) \times 1}}$$

Δηλαδη:

$$\hat{\mathbf{y}} = \Phi \mathbf{w}$$

• Θελουμε:

$$\hat{\mathbf{y}} \approx \mathbf{y} \iff \Phi \mathbf{w} \approx \mathbf{y}$$

Ομως $\mathbf{y} = \hat{\mathbf{y}} + \mathbf{e}$. Για να λυσει λοιπον αυτο το προβλημα το Matlab, μεσω της εντολης `w=Phi \y`, το μετατρεπει σε προβλημα Least Squares:

$$\min_{\mathbf{w}} \|\Phi \mathbf{w} - \mathbf{y}\|^2$$

Αυτο λυνεται μαθηματικα ως εξης:

– Θελουμε να ελαχιστοποιησουμε:

$$J(\mathbf{w}) = \|\Phi \mathbf{w} - \mathbf{y}\|_2^2 = (\Phi \mathbf{w} - \mathbf{y})^\top (\Phi \mathbf{w} - \mathbf{y})$$

– Παραγωγιζουμε ως προς $\mathbf{w}$ και θετουμε τη παραγωγο ιση με μηδεν:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = 2\Phi^\top (\Phi \mathbf{w} - \mathbf{y}) = 0$$

– Απο οπου προκυπτουν οι κανονικες εξισωσεις (normal equations)

$$\Phi^\top \Phi \mathbf{w} = \Phi^\top \mathbf{y}$$

– Αν ο $\Phi^T \Phi$ είναι αντιστρεψιμος, τότε:

$$\mathbf{w}^* = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}$$

– Αλλιως:

$$\mathbf{w}^* = \Phi^+ \mathbf{y}$$

οπου Φ^+ είναι ο Moore-Penrose ψευδοαντιστροφος.

Ωστοσο, το Matlab λυνει το Least Squares προβλημα με αλλες με-
θοδους, οπως QR-Decomposition ή SVD-Decomposition, για να
αποφυγει τον υπολογισμο του αντιστροφου πινακα.

**Συνοψιζοντας, η πορεία που ακολουθησα στον αλγοριθμο μου
ειναι η εξης:**

- Δημιουργω αρχικο πληθυσμο
- Δινω τον αρχικο πληθυσμο (θ) στη fitness
- Για καθε ατομο θ :
 - Χτιζει τον πινακα $\Phi(\theta)$
 - Λυνει LS για να βρει τα $w(\theta)$
 - Υπολογιζει $\text{fitness} = \text{MSE} + \text{penalty}$
 - Επιστρεφει ενα διανυσμα fitness (ενα στοιχειο ανα ατομο με τη fitness value του)
- Κραταω το καλυτερο (ελαχιστο) fitness value και το αντι-στοιχο θ
- Φτιαχνω νεο πληθυσμο με elitism + selection + crossover + mutation
- Ξαναυπολογιζω fitness στον νεο πληθυσμο
- Ενημερωνω best-so-far θ και fitness

- Στο τέλος κρατάω το καλύτερο θ που βρεθηκε σε όλες τις γενιες
- Εχοντας βρει το καλύτερο θ , βρίσκω τα βάρη για αυτό, και υπολογίζω τις "προβλεψεις", δηλαδή τις εξόδους της προσεγγιστικής συναρτησης, για τα training/validation/test data. Με βάση αυτά, τυπώνω τα MSE, κάνω Scatter plot ($\hat{y}$ vs y (ιδανικά θέλουμε τα σημεία να βρίσκονται πάνω στην $y=x$), και 3D graph της προσεγγιστικής συναρτησης και της πραγματικής.

4 Αποτελέσματα

Όλα τα πειράματα, έγιναν με σταθερό seed (rng(1)) ώστε το πρόγραμμα να παράγει τα ίδια αποτελέσματα, και οποιαδήποτε αλλαγή κάνω να ξέρω ότι η διαφορετική έξοδος οφείλεται μόνο σε αυτή. Αφού πήρα το βελτιστό μοντέλο μου, έβγαλα το seed, για να το αξιολογήσω πιο γενικά. Οι διαφορές που θα φανούν στην έξοδο, θα οφείλονται στην τυχαιότητα του γενετικού αλγορίθμου (selection, crossover, mutation) αλλά και στα διαφορετικά δεδομένα που δημιουργούνται σε κάθε run. Να σημειώσω ότι η αναλογία δεδομένων που δημιουργήσα είναι:

- Training Data: 600
- Validation Data: 200
- Test Data: 200

Ετρέξα τον αλγόριθμο 10 φορές για 800 generations, και τα αποτελέσματα που πήρα ήταν αυτά:

Run	Train MSE	Val MSE	Test MSE	Active Gaussian
1	0.000298	0.000324	0.000422	2
2	0.000425	0.000762	0.000467	2
3	0.000363	0.000382	0.000427	2
4	0.000300	0.000193	0.000396	2
5	0.000430	0.000271	0.000315	3
6	0.000324	0.000552	0.000380	2
7	0.000284	0.000339	0.000372	2
8	0.000285	0.000398	0.000328	2
9	0.000294	0.000877	0.000757	2
10	0.000324	0.000705	0.000324	2
Avg	0.0003327	0.0004803	0.0004188	2

Παρατηρούμε ότι σε όλα τα runs εκτός από ένα, οι ενεργές Γκαουσιανές ήταν πάντα 2. Αυτό που συνέβη στο 5ο run, είναι ότι:

$$\text{MSE}_5 + \lambda \cdot 3 < \text{MSE}'_5 + \lambda \cdot 2$$

Δηλαδή ο αλγόριθμος προτίμησε να πάρει κάποιο θ που δίνει 3 ενεργές Γκαουσιανές, διότι πέτυχε τόσο χαμηλό MSE, ώστε το άθροισμα του MSE και της ποινής, δηλαδή η fitness, να είναι μικρότερο της fitness που πετυχαίνουν όλα τα θ που δίνουν 2 ενεργές Γκαουσιανές.

Το output του προγράμματος με το seed που χρησιμοποιούσα φαίνεται παρακάτω. Οι γενιές τυπώνονται ανά 40.

4.1 Output

Command Window

```
Generation 40/800 | best fitness value = 0.002625
Generation 80/800 | best fitness value = 0.002037
Generation 120/800 | best fitness value = 0.001967
Generation 160/800 | best fitness value = 0.001806
Generation 200/800 | best fitness value = 0.001703
Generation 240/800 | best fitness value = 0.001661
Generation 280/800 | best fitness value = 0.001625
Generation 320/800 | best fitness value = 0.001576
Generation 360/800 | best fitness value = 0.001554
Generation 400/800 | best fitness value = 0.001530
Generation 440/800 | best fitness value = 0.001497
Generation 480/800 | best fitness value = 0.001474
Generation 520/800 | best fitness value = 0.001448
Generation 560/800 | best fitness value = 0.001442
Generation 600/800 | best fitness value = 0.001414
Generation 640/800 | best fitness value = 0.001399
Generation 680/800 | best fitness value = 0.001394
Generation 720/800 | best fitness value = 0.001390
Generation 760/800 | best fitness value = 0.001386
Generation 800/800 | best fitness value = 0.001386
```

```
=== RESULTS ===
```

```
Train MSE: 0.000186
```

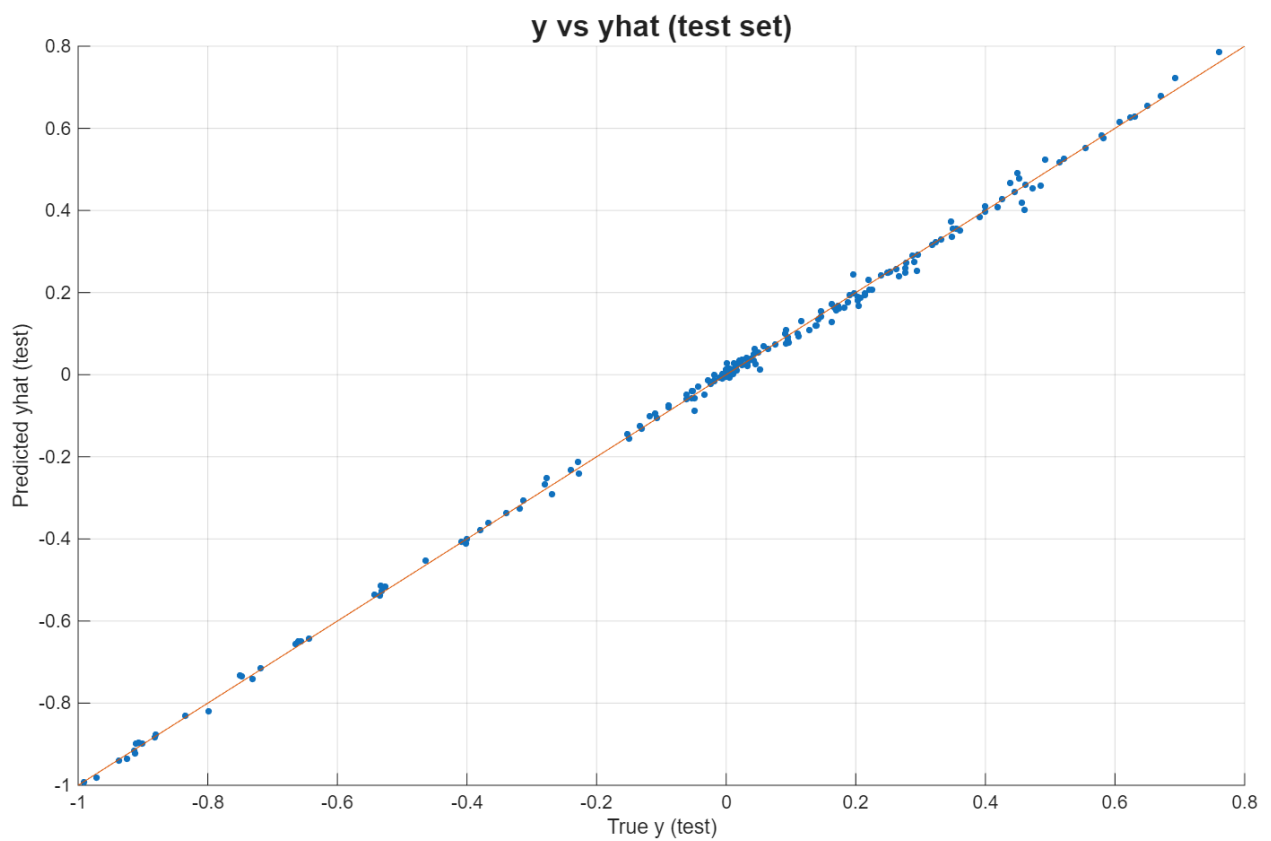
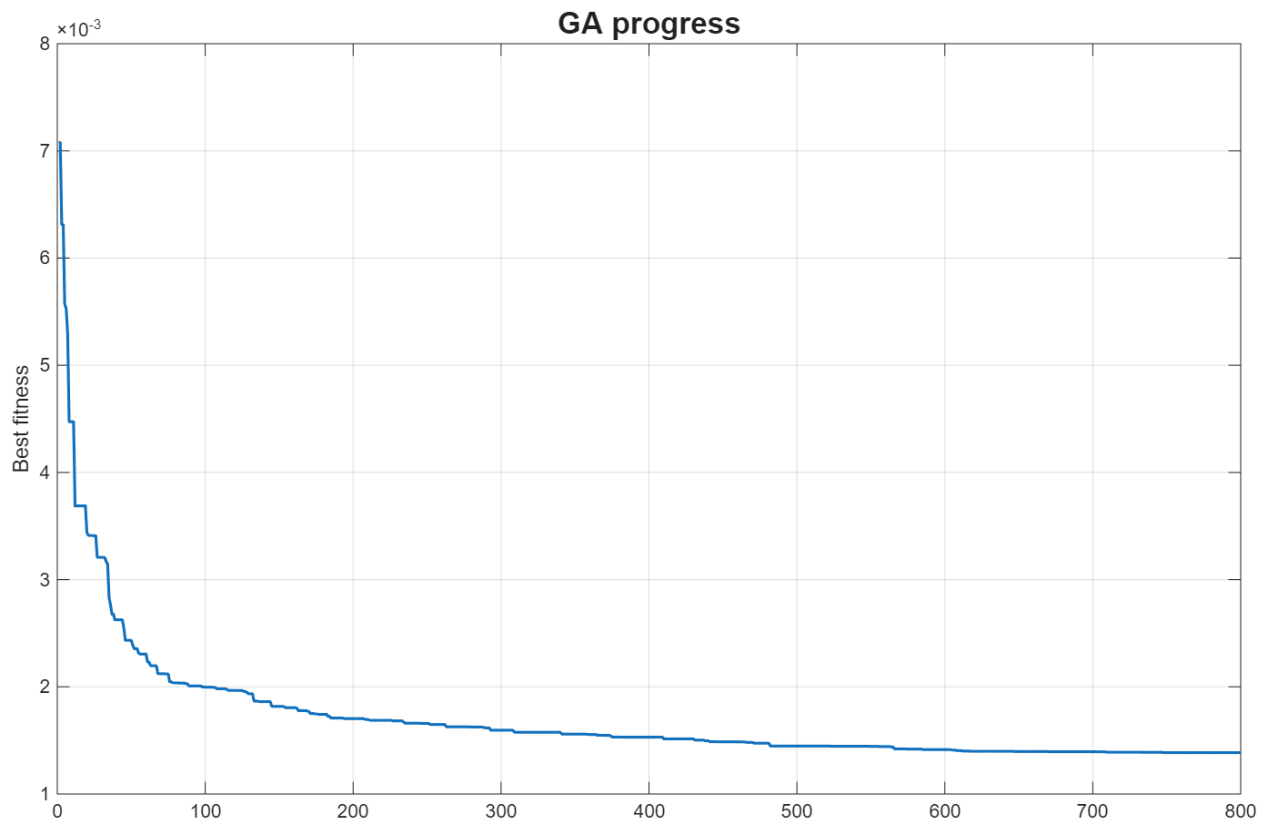
```
Validation MSE: 0.000199
```

```
Test MSE: 0.000214
```

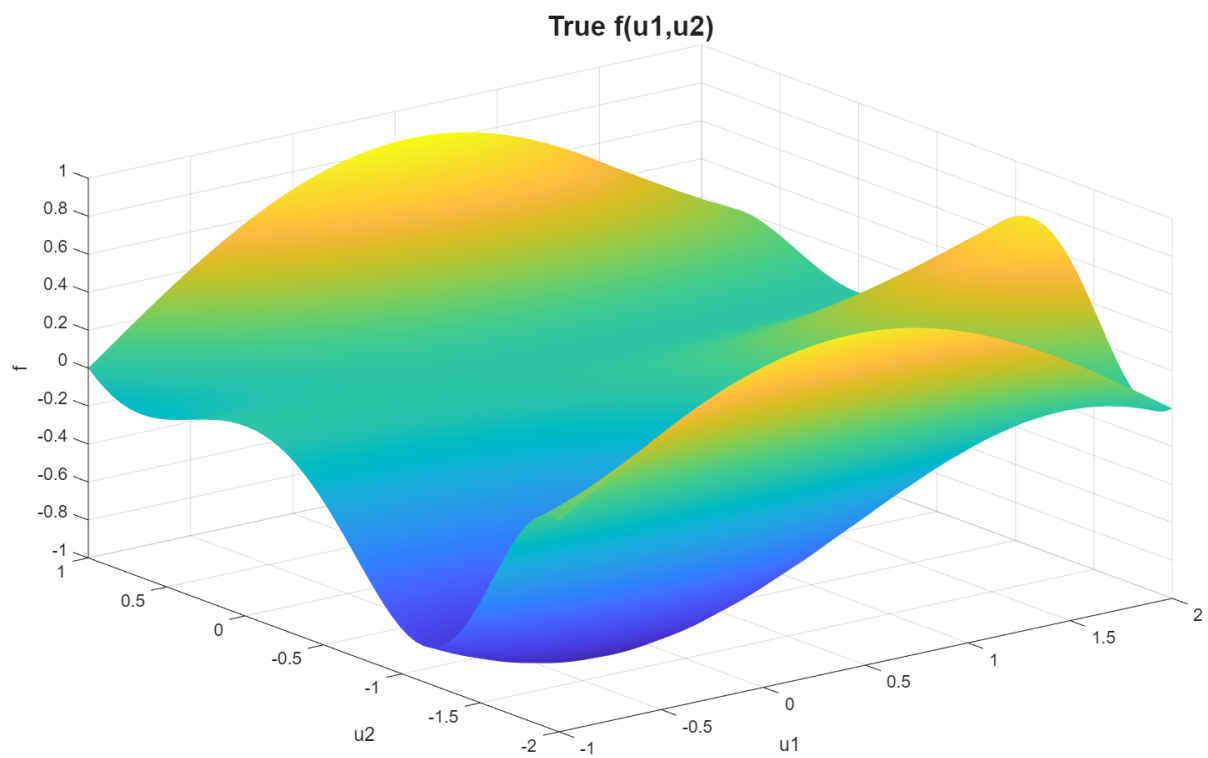
```
Active Gaussians: 4 / 15
```

Βλεπουμε 4 ενεργες Γκαουσιανες

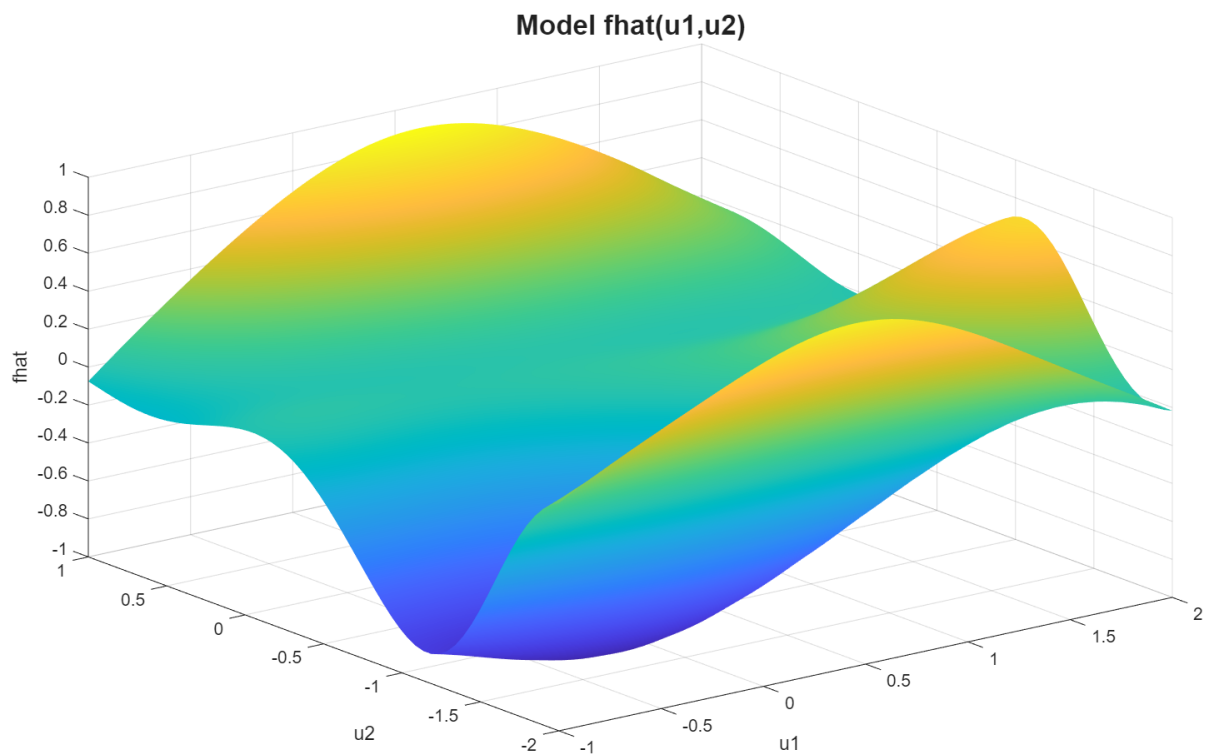
4.2 Fitness Progress on Training Data - Scatter Plot on Test set



4.3 f -- 3D Graph



4.4 Προσεγγιστική $\hat{f}$ -- 3D graph



Απο ενδιαφερον και μονο, βρηκα τα θ και τα βαρη των ενεργων Γκαουσιανων:

```
Gaussian 8:  
  center = (0.465, -0.529)  
  sigma  = (1.221, 1.254)  
  weight  = 50.765  
  
Gaussian 2:  
  center = (0.210, -0.254)  
  sigma  = (1.143, 1.019)  
  weight  = -25.158  
  
Gaussian 10:  
  center = (0.759, -0.209)  
  sigma  = (1.104, 1.067)  
  weight  = -20.123  
  
Gaussian 9:  
  center = (0.444, -1.505)  
  sigma  = (1.171, 0.760)  
  weight  = -19.813  
  
Bias: w0=-1.711
```

οπου $\text{center} = (c_1, c_2)$, $\text{sigma} = (\sigma_1, \sigma_2)$, σχεδιασα την προσεγγιστικη $\hat{f}(u_1, u_2) = w_0 + \sum_{\text{active}} w_{\text{active}} G_{\text{active}}$ στο Desmos, και πραγματι βγηκε η ιδια:

